FlowStock

Complete System Documentation

Document Type: Technical Architecture & System Reference

Audience: Development Team

Date: April 2026

Status: Reflects production codebase — no assumed content

FlowStock — Complete System Documentation

Document Type: Technical Architecture & System Reference

Audience: Development Team

Date: April 2026

Status: Reflects production codebase as-is — no placeholder or assumed content

1. Project Overview

FlowStock is a multi-tenant B2B inventory and logistics SaaS platform built for Indian businesses. It enables:

- Admins to manage inventory, process orders, manage delivery drivers, and view business analytics
- Customers to browse products, place orders, track deliveries, and file complaints
- Drivers to manage and complete assigned deliveries with OTP-based verification
- Super Admins to oversee the entire platform, monitor system health, and manage bug reports

The system is fully multi-tenant — multiple independent businesses run on the same platform without data leakage.

2. Technology Stack

%, Frontend

Technology	Version	Purpose
React	19.x	UI framework
React Router DOM	7.x	Client-side routing
Recharts	3.x	Analytics charts
@supabase/supabase-js	2.x	Auth and DB client

Vite	—	Build tool / dev server

%¸ Backend

Technology	Version	Purpose
Node.js	24.x	Runtime (ESM modules)
Express	4.x	HTTP server framework
@supabase/supabase-js	2.x	DB + Auth via service-role client
PDFKit	0.18.x	Invoice PDF generation
Twilio	5.x	WhatsApp notification delivery
dotenv	16.x	Environment variable loading
cors	2.x	CORS middleware

%¸ Database & Infrastructure

Service	Purpose
Supabase (PostgreSQL)	Primary database (RLS-enabled)
Supabase Auth	Google OAuth + JWT issuance
Supabase Storage	File storage — logos, images, screenshots
Twilio WhatsApp API	Order status notifications
Google Maps Directions API	Route optimization (Tier 1)
OpenRouteService API	Route optimization fallback (Tier 2)

3. System Architecture

%¸ High-Level Overview

```
CLIENT (Vite SPA)
  React + React Router — 4 portal UIs
  adminApi / customerApi / driverApi / devApi
         |
         | HTTP / HTTPS
         v
EXPRESS SERVER (Node.js)
  registerRoutes() — 11 route modules
  CORS -> JSON -> Routes -> Global Error Handler
         |                    |
         v                    v
  Supabase DB (PostgreSQL)    Supabase Storage Buckets
  14 tables, RLS-enabled      business-logos
                              product-images
                              complaint-images
                              bug-screenshots
```

%¸ Request Lifecycle

```
User Action
  -> API client adds Authorization + x-business-id headers
    -> Express route
      -> Auth middleware validates token (JWT or driver session)
        -> resolveBusinessId validates + attaches business context
          -> Route handler queries supabaseAdmin (bypasses RLS)
            -> JSON response -> React state update -> UI re-render
```

%¸ Monorepo Structure

```
FlowStock/
  client/
    src/
      App.jsx               Route tree
      components/           ProtectedRoute, SuperAdminRoute, BugReportModal
      context/              AuthContext.tsx, DriverContext.jsx
      layouts/              AdminLayout, CustomerLayout, DriverLayout, DevLayout
      lib/                  adminApi, customerApi, driverApi, devApi, supabase
      pages/                admin/, customer/, driver/, dev/, auth pages
  server/
    src/
      index.js              Express entry point
      middleware/           6 middleware modules
      routes/               11 route modules
      services/             notificationService, inventoryService
    lib/
      supabase.js           Service-role client (supabaseAdmin)
      supabaseAnon.js       Anon-key client (JWT validation only)
  supabase/sql/             34 migration scripts
```

4. Authentication & Session Management

%¸ 4.1 Admin & Customer Authentication (Google OAuth via Supabase)

Sign-in flow:

1. User clicks "Sign in with Google"
2. Supabase Auth returns JWT access_token (signed by Supabase)

3. On first sign-in: DB trigger handle_new_user inserts profiles row with role = 'customer'
4. Admin / super_admin roles set manually in Supabase Table Editor or via SQL
5. Frontend fetches role from profiles table and redirects based on role

Role to Dashboard Mapping:

Role	Redirect Path
admin	/admin/dashboard
customer	/customer/home
driver	/driver/dashboard
super_admin	/dev/dashboard

Per-request validation:

```
Authorization: Bearer <token>
  -> supabaseAnon.auth.getUser(token)   validates signature + expiry
  -> supabaseAdmin.from("profiles").select("role")
  -> attaches req.authUser, req.authProfile to request
```

%¸ 4.2 Driver Authentication (Custom Token System)

Drivers do NOT use Google OAuth. They use phone + password.

Login flow:

1. POST /api/driver/auth/login — phone + password
2. Server normalises phone to E.164 (+91XXXXXXXXXX)
3. Verifies bcrypt password_hash from drivers table
4. Generates 64-char random hex token via crypto.randomBytes
5. Inserts into driver_sessions (expires_at = now() + 30 days)

6. Returns { token, driver, businesses[] }
7. Client stores in localStorage("driverToken")

Per-request validation:

```
Authorization: Bearer <driverToken>
  -> driver_sessions lookup by token
  -> Checks expires_at > now()
  -> Loads driver row from drivers table
  -> Validates x-business-id against driver_business_links
  -> Attaches req.driverUser, req.businessId
```

---

5. Role-Based Access Control (RBAC)

%¸ 5.1 Roles

Role	How Assigned	Portal
customer	Auto via DB trigger on signup	/customer/*
admin	Set manually in Supabase	/admin/*
driver	Created in custom drivers table	/driver/*
super_admin	Set manually in Supabase	/dev/*

%¸ 5.2 Frontend Route Guards

Component	Protects	Check
ProtectedRoute	All authenticated routes	user !== null via AuthContext
DriverProtectedRoute	/driver/*	driverToken present via DriverContext
SuperAdminRoute	/dev/*	role === 'super_admin' via AuthContext
RouteErrorBoundary	All routes	Catches render errors

%¸ 5.3 Backend Middleware

Middleware	Validates
requireAuth	Supabase JWT — any authenticated user
requireAdmin	Supabase JWT + profiles.role = 'admin'
requireDriverAuth	Driver session token + business membership
superAdminOnly	Supabase JWT + profiles.role = 'super_admin'
resolveBusinessId	x-business-id header validated against DB
apiLogger	Fire-and-forget request logging to api_logs table

%¸ 5.4 Middleware by Route Group

```
/api/admin/*      requireAdmin (entire router)
/api/customer/*   requireAuth + resolveBusinessId (entire router)
/api/driver/auth/* no middleware (login endpoint)
/api/driver/*     requireDriverAuth (entire router)
/api/dev/*        superAdminOnly (entire router)
/api/bugs/*       authenticateAny (driver token OR Supabase JWT)
/api/business/*   no auth (join flow uses business_code)
/api/complaints/* POST: requireAuth, GET: requireAdmin
/api/orders/*     requireAdmin (entire router)
/api/inventory/*  requireAdmin (entire router)
```

6. Frontend Pages

%¸ Admin Portal (/admin/*)

Route	File	Description
/admin/dashboard	Dashboard.jsx	KPI cards, recent orders, charts

/admin/inventory	Inventory.jsx	Inventory CRUD with image upload
/admin/orders	Orders.jsx	Order management, status, driver assignment
/admin/drivers	Drivers.jsx	Driver management, creation, history
/admin/reports	Reports.jsx	Revenue + sales reports
/admin/complaints	Complaints.jsx	Customer complaint management
/admin/business-setup	BusinessSetup.jsx	Business profile + logo upload

%¸ Customer Portal (/customer/*)

Route	File	Description
/customer/home	Home.jsx	Product shop, cart, checkout modal
/customer/orders	Orders.jsx	Order history + status tracking
/customer/profile	Profile.jsx	Profile + saved delivery addresses
/customer/support	Support.jsx	File and view complaints

%¸ Driver Portal (/driver/*)

Route	File	Description
/driver/dashboard	Dashboard.jsx	Active deliveries + today's tasks
/driver/history	History.jsx	Past completed deliveries
/driver/profile	Profile.jsx	Driver profile + vehicle info
/driver/select-business	SelectBusiness.jsx	Choose active business
/driver/setup	Setup.jsx	First-time profile setup
/driver/login	Login.jsx	Phone + password login

%¸ Dev Console (/dev/*)

Route	File	Description
/dev/dashboard	Overview.jsx	Platform-wide KPIs
/dev/businesses	Businesses.jsx	All businesses + detail panel
/dev/customers	Customers.jsx	All customers platform-wide
/dev/products	Products.jsx	All inventory platform-wide
/dev/drivers	Drivers.jsx	All drivers platform-wide

/dev/analytics	Analytics.jsx	4 recharts charts with date filter
/dev/system-health	SystemHealth.jsx	API logs, error rates, slow endpoints
/dev/bugs	BugReports.jsx	Bug report management console

7. Backend Route Modules

Module	Mount Path	Auth	Lines
admin.js	/api/admin	requireAdmin	1400
customerOrders.js	/api/customer	requireAuth + resolveBusinessId	720
driverAuth.js	/api/driver/auth	none / requireDriverToken	340
driverRoutes.js	/api/driver	requireDriverAuth	1100
orders.js	/api/orders	requireAdmin	2100
inventory.js	/api/inventory	requireAdmin	350
businessSetup.js	/api/business	none / requireAuth	310
complaints.js	/api/complaints	mixed	370
bugs.js	/api/bugs	authenticateAny	156
dev.js	/api/dev	superAdminOnly	854

8. Database Schema

%¸ profiles

Column	Type	Notes
id	uuid	PK, FK to auth.users ON DELETE CASCADE
role	text	CHECK IN ('admin' 'customer' 'driver' 'super_admin')
display_name	text	nullable
updated_at	timestamptz	DEFAULT now()

Trigger handle_new_user inserts role = 'customer' on every new auth signup.

%¸ business_profile

Column	Type	Notes
id	uuid	PK
admin_id	uuid	FK to auth.users, UNIQUE
business_name	text	NOT NULL
owner_name	text	NOT NULL
phone	text	NOT NULL
email	text	nullable
address	text	nullable
warehouse_location	text	nullable
business_type	text	Retail / Wholesale / Distributor / Manufacturer
gst_number	text	nullable
logo_url	text	nullable
brand_color	text	DEFAULT '#6366f1'
updates_phone	text	NOT NULL — receives WhatsApp alerts
business_code	text	Unique alphanumeric join code
created_at	timestamptz	DEFAULT now()
updated_at	timestamptz	DEFAULT now()

%¸ user_businesses

Column	Type	Notes
id	uuid	PK
user_id	uuid	FK to auth.users ON DELETE CASCADE
business_id	uuid	FK to business_profile ON DELETE CASCADE
role	text	CHECK IN ('admin','customer','driver')
created_at	timestamptz	DEFAULT now()

UNIQUE constraint: (user_id, business_id)

%¸ inventory_items

Column	Type	Notes
id	uuid	PK
name	text	NOT NULL
stage	text	raw, processing, packaged, ready, draft, archived
quantity	numeric	DEFAULT 0, CHECK >= 0
low_stock_alert	numeric	DEFAULT 0
unit_price	numeric	nullable
unit	text	nullable (e.g. "kg", "box")
description	text	nullable
image_url	text	nullable
image_urls	text[]	DEFAULT '{}'
admin_id	uuid	FK to profiles
business_id	uuid	FK to business_profile
updated_at	timestamptz	DEFAULT now()

%¸ orders

Column	Type	Notes
id	uuid	PK
customer_id	uuid	FK to profiles
driver_id	uuid	nullable
business_id	uuid	FK to business_profile
region	text	Delivery area
status	text	pending, confirmed, dispatched, out_for_delivery, delivered, rejected
final_total	numeric	nullable
discount_amount	numeric	nullable
guest_customer_name	text	nullable
notes	text	nullable
delivery_otp	text	nullable (4-digit)
otp_verified	boolean	DEFAULT false
delivered_at	timestamptz	nullable
confirmed_at	timestamptz	nullable

dispatched_at	timestamptz	nullable
out_for_delivery_at	timestamptz	nullable
cancelled_at	timestamptz	nullable
customer_cancel_proof_url	text	nullable
stock_deducted	boolean	DEFAULT false
created_at / updated_at	timestamptz	DEFAULT now()

%¸ order_line_items

Column	Type	Notes
id	uuid	PK
order_id	uuid	FK to orders ON DELETE CASCADE
inventory_item_id	uuid	nullable
item_name	text	NOT NULL (snapshot at order time)
quantity	numeric	NOT NULL, CHECK > 0
unit_price	numeric	NOT NULL, CHECK >= 0

%¸ drivers

Column	Type	Notes
id	uuid	PK
phone	text	NOT NULL UNIQUE
password_hash	text	NOT NULL (bcrypt)
name	text	nullable
vehicle_details	text	nullable
license_number	text	nullable
is_profile_complete	boolean	DEFAULT false
is_active	boolean	DEFAULT true
created_at / updated_at	timestamptz	DEFAULT now()

%¸ driver_sessions

Column	Type	Notes

id	uuid	PK
driver_id	uuid	FK to drivers ON DELETE CASCADE
token	text	NOT NULL UNIQUE (64-char hex)
expires_at	timestamptz	now() + 30 days
created_at	timestamptz	DEFAULT now()

%¸ driver_business_links

Column	Type	Notes
id	uuid	PK
driver_id	uuid	FK to drivers ON DELETE CASCADE
business_id	uuid	FK to business_profile ON DELETE CASCADE
created_at	timestamptz	DEFAULT now()

UNIQUE constraint: (driver_id, business_id)

%¸ deliveries

Column	Type	Notes
id	uuid	PK
order_id	uuid	FK to orders
driver_id	uuid	FK to drivers
business_id	uuid	FK to business_profile
status	text	pending, active, completed, failed
delivered_at	timestamptz	nullable

%¸ complaints

Column	Type	Notes
id	uuid	PK
customer_id	uuid	FK to profiles
business_id	uuid	nullable
order_id	uuid	nullable

message	text	NOT NULL, CHECK length >= 10
issue_type / product_name	text	nullable
image_urls	text[]	nullable
status	text	CHECK IN ('open','in_review','resolved')
admin_comment / resolution_message	text	nullable
resolved_at	timestamptz	nullable
created_at / updated_at	timestamptz	DEFAULT now()

%¸ customer_addresses

Column	Type	Notes
id	uuid	PK
customer_id	uuid	FK to profiles
label	text	e.g. "Home", "Office"
region	text	Area name
address	text	Full address
pincode	text	nullable
is_default	boolean	DEFAULT false
created_at	timestamptz	DEFAULT now()

%¸ bug_reports

Column	Type	Notes
id	uuid	PK
title / category / severity	text	NOT NULL
description	text	NOT NULL
image_urls	text[]	DEFAULT '{}'
device_info	jsonb	browser, screen, user agent, timestamp
page_url	text	Page where bug reported
reported_by_type	text	CHECK IN ('admin','customer','driver')
reported_by_id	uuid	Verified server-side
business_id	uuid	nullable

status	text	CHECK IN ('open' 'in_review' 'resolved' 'wont_fix')
super_admin_note	text	Internal only — never exposed to reporter
resolved_at	timestamptz	nullable
created_at / updated_at	timestamptz	DEFAULT now()

%¸ api_logs

Column	Type	Notes
id	uuid	PK
method	text	HTTP method
endpoint	text	Route path
status_code	int	HTTP response code
response_time_ms	int	nullable
business_id	uuid	nullable
error_message	text	nullable
created_at	timestamptz	DEFAULT now()

%¸ Entity Relationships

```
auth.users
  1:1  profiles (role)
  1:1  business_profile (admin_id)
  M:M  user_businesses <-> business_profile

business_profile
  1:N  inventory_items
  1:N  orders
  1:N  deliveries
  1:N  api_logs
  M:M  driver_business_links <-> drivers

orders
  1:N  order_line_items
  1:1  deliveries

drivers
  1:N  driver_sessions
  M:M  driver_business_links

profiles
  1:N  orders (as customer)
  1:N  complaints
  1:N  customer_addresses
```

9. API Documentation

%¸ /api/admin/* — Admin Portal (requireAdmin)

Method	Endpoint	Description
GET	/api/admin/users	List all users
POST	/api/admin/create-user	Create user in Supabase Auth
GET	/api/admin/reports	Revenue, orders, top products
GET	/api/admin/dashboard-stats	KPI cards
GET	/api/admin/drivers	All drivers for this business
GET	/api/admin/drivers/search	Search drivers
POST	/api/admin/drivers/create	Create driver account
POST	/api/admin/drivers/add-existing	Link existing driver
DELETE	/api/admin/drivers/:id/remove	Unlink driver
POST	/api/admin/drivers/link	Link driver by phone
GET	/api/admin/drivers/:id/deliveries	Driver delivery history
POST	/api/admin/auto-assign-driver	Auto-assign least-loaded driver
GET	/api/admin/dispatch-eta	Calculate delivery ETA
GET	/api/admin/business-profile	Get business profile
GET	/api/admin/my-businesses	All businesses this admin belongs to
PUT	/api/admin/business-profile	Create or update business profile
POST	/api/admin/business-profile/logo	Upload business logo

%¸ /api/orders/* — Order Management (requireAdmin)

Method	Endpoint	Description
GET	/api/orders	All orders for this business
POST	/api/orders	Create new order + line items
GET	/api/orders/:id	Single order full detail
PUT	/api/orders/:id	Update order

PUT	/api/orders/:id/status	Change status (triggers WhatsApp)
PUT	/api/orders/:id/assign-driver	Assign driver + create delivery
GET	/api/orders/:id/invoice	Generate PDF invoice
POST	/api/orders/:id/verify-delivery	OTP delivery verification
POST	/api/orders/auto-confirm	Bulk-confirm eligible orders
POST	/api/orders/bulk-action	Bulk status change

%¸ /api/inventory/* — Inventory (requireAdmin)

Method	Endpoint	Description
GET	/api/inventory	All inventory items
GET	/api/inventory/:id/insights	Demand insights + restock suggestions
POST	/api/inventory	Create item
PUT	/api/inventory/:id	Update item
DELETE	/api/inventory/:id	Delete item
POST	/api/inventory/upload-image	Upload product image

%¸ /api/customer/* — Customer Portal (requireAuth + resolveBusinessId)

Method	Endpoint	Description
GET	/api/customer/products	Products visible to customer
GET	/api/customer/business-info	Business name, logo, branding
POST	/api/customer/orders	Place new order
GET	/api/customer/orders	All customer orders
PUT	/api/customer/orders/:id/cancel	Cancel order
GET	/api/customer/profile	Customer profile
PUT	/api/customer/profile	Update display name
GET	/api/customer/addresses	List saved addresses
POST	/api/customer/addresses	Add address
PUT	/api/customer/addresses/:id	Update address
DELETE	/api/customer/addresses/:id	Delete address

%¸ /api/driver/auth/* — Driver Auth

Method	Endpoint	Auth	Description
POST	/api/driver/auth/login	None	Phone + password -> token
PUT	/api/driver/auth/setup	Token	First-time setup
GET	/api/driver/auth/me	Token	Current driver + businesses
POST	/api/driver/auth/logout	Token	Delete session
PUT	/api/driver/auth/change-password	Token	Change password

%¸ /api/driver/* — Driver Operations (requireDriverAuth)

Method	Endpoint	Description
GET	/api/driver/businesses	Businesses driver belongs to
GET	/api/driver/deliveries	Active deliveries
PUT	/api/driver/deliveries/:id/start	Mark delivery started
POST	/api/driver/deliveries/:id/verify-otp	Submit OTP to verify delivery
POST	/api/driver/deliveries/:id/complete	Mark completed
GET	/api/driver/history	Past deliveries
PUT	/api/driver/profile	Update driver profile
POST	/api/driver/optimize-route	3-tier route optimization

%¸ /api/business/* — Business Join

Method	Endpoint	Description
GET	/api/business/info/:code	Business info by join code (public)
POST	/api/business/setup	Create new business
POST	/api/business/join	Join business by code
GET	/api/business/my-businesses	All businesses user belongs to

%¸ /api/complaints/*

Method	Endpoint	Auth	Description
POST	/api/complaints	requireAuth	Submit complaint

GET	/api/complaints/mine	requireAuth	Customer's complaints
GET	/api/complaints	requireAdmin	All complaints for this business
PUT	/api/complaints/:id/resolve	requireAdmin	Resolve complaint

%¸ /api/bugs/* — Bug Reporting (authenticateAny)

Method	Endpoint	Description
POST	/api/bugs/upload-screenshot	base64 JSON -> Supabase Storage
POST	/api/bugs/report	Submit bug report

%¸ /api/dev/* — Dev Console (superAdminOnly)

Method	Endpoint	Description
GET	/api/dev/overview	Platform-wide KPIs
GET	/api/dev/businesses	All businesses with revenue
GET	/api/dev/business/:id/detail	Deep business detail
GET	/api/dev/customers	All customers
GET	/api/dev/products	All inventory
GET	/api/dev/drivers	All drivers
GET	/api/dev/analytics	Time-series data (?days=7/30/90)
GET	/api/dev/system-health	API log summary
GET	/api/dev/system/stats	Aggregated log stats
GET	/api/dev/system/errors	Recent errors
GET	/api/dev/system/slow-endpoints	Slowest endpoints last 24h
GET	/api/dev/bugs/count	Open bug count
GET	/api/dev/bugs	All bug reports with filters
GET	/api/dev/bugs/:id	Full bug detail + super_admin_note
PUT	/api/dev/bugs/:id	Update status + internal note

10. Core Features

%¸ Multi-Business System

- Every user can belong to multiple businesses via user_businesses
- Active business tracked in localStorage("selectedBusinessId")
- Each business has unique alphanumeric business_code
- Join link: /join/<business_code>
- x-business-id header validated per request by resolveBusinessId middleware

%¸ Order Lifecycle

```
pending -> confirmed -> dispatched -> out_for_delivery -> delivered
               |                          |
               v                          v
          rejected                   cancelled
```

Each status change: updates timestamp column + triggers WhatsApp notification + deducts stock at confirmed (stock_deducted flag prevents double-deduction).

%¸ Driver Route Optimization (3-Tier)

Tier	Method	Condition
1	Google Maps Directions API	GOOGLE_MAPS_API_KEY configured
2	OpenRouteService	ORS_KEY configured + Google failed
3	Nearest-Neighbour Heuristic (built-in)	Always available

Returns: ordered stops, total_km, total_min, Google Maps URL, fuel estimate.

%¸ OTP Delivery Verification

1. delivery_otp (4-digit) stored on order when driver assigned
2. Customer tells OTP to driver at delivery
3. Driver submits via POST /api/driver/deliveries/:id/verify-otp

4. Server validates -> otp_verified = true

%¸ Bug Reporting System

- BugReportModal.jsx embedded in all 3 portals
- Captures: title, category, severity, description, up to 3 screenshots
- Auto-captures: browser, screen size, page URL, timestamp
- Screenshots as base64 JSON (no multipart) -> decoded and stored in Supabase Storage
- Reporter identity enforced server-side (never from request body)

---

11. Data Isolation

%¸ Admin Isolation

All queries scoped by admin_id or business_id of authenticated user. Cannot access another business's data.

%¸ Customer Isolation

Membership verified via user_businesses on every request. Orders and addresses scoped by customer_id.

%¸ Driver Isolation

driver_business_links validated per request. Deliveries scoped to driver_id + business_id.

%¸ Anti-Spoofing

resolveBusinessId middleware checks user_businesses membership before allowing access. Returns 403 if not a member — zero data returned.

---

12. Security Design

%¸ Authentication Threats & Controls

Threat	Control
JWT forgery	supabaseAnon.auth.getUser() cryptographic verification
Driver token forgery	Random 64-char hex validated against driver_sessions
Session replay after logout	Logout deletes session row — token instantly invalid
Expired driver sessions	expires_at checked per request; row deleted on expiry
Role self-elevation	profiles.role checked server-side; clients can't write it

%¸ Authorization Threats & Controls

Threat	Control
Customer accessing admin routes	requireAdmin checks profiles.role = 'admin'
Admin accessing dev routes	superAdminOnly checks profiles.role = 'super_admin'
Role oracle attack on dev console	All failure cases return identical 403 { error: "Access denied" }
Bug report identity spoofing	Reporter ID from req.authId (server-verified), never from body

%¸ Input Validation

- All required fields validated server-side
- ENUMs validated against explicit allowlists
- UUIDs validated via regex before DB queries
- 20MB JSON body limit at Express level
- All file uploads require authentication

13. Error Handling & Logging

%¸ Global Error Handler

```
app.use((err, req, res, next) => {
  console.error("[GLOBAL ERROR]", err);
  if (res.headersSent) return;
  res.status(500).json({ success: false, error: "Internal Server Error" });
});
```

Registered last in Express. Guarantees JSON response for all unhandled errors.

%¸ Standard Response Format

Type	Format
Success	{ "success": true, "data": {...} }
Error	{ "success": false, "error": "message" }
List	{ "success": true, "data": [...], "count": N }

%¸ API Logger

Hooks to res.on("finish"). Inserts to api_logs: method, endpoint, status_code, response_time_ms, business_id, error_message. Never blocks responses.

14. External Integrations

%¸ Twilio (WhatsApp)

Detail	Value
Package	twilio@5.x (lazy-loaded)
Env vars	TWILIO_ACCOUNT_SID, TWILIO_AUTH_TOKEN, TWILIO_WHATSAPP_FROM
Purpose	Order status + complaint notifications
Failure mode	Logs warning, skips — no crash

%¸ Google Maps Directions API

Detail	Value
Env var	GOOGLE_MAPS_API_KEY
Purpose	Multi-stop route optimization (Tier 1)
Timeout	10 seconds

%¸ OpenRouteService API

Detail	Value
Env var	ORS_KEY
Purpose	Route optimization fallback (Tier 2)
Timeout	10 seconds

%¸ Supabase Storage Buckets

Bucket	Contents	Public
business-logos	Business logo images	Yes
product-images	Inventory item images	Yes
complaint-images	Customer complaint photos	Yes
bug-screenshots	Bug report screenshots	Yes

Document generated from full source code analysis — April 2026

FlowStock — Internal Technical Reference

No code was modified during the generation of this document.